

Principles of Programming Languages, 2026.02.18

Important notes

- Total available time: 2h (*multichance* students do not need to solve Exercise 3).
- You may use any written material you need, and write in English or in Italian.
- You cannot use electronic devices during the exam: every phone must be turned off and kept on your table.
- You cannot use library functions not covered in class in your code.
- Each exercise is valued 10 points, +1 bonus.

Exercise 1, Scheme

Consider the **choose/fail** non-deterministic construct seen in class and reported here:

```
(define *paths* '())

(define (choose choices)
  (if (null? choices)
      (fail)
      (call/cc
        (lambda (cc)
          (set! *paths*
                (cons (lambda ()
                        (cc (choose (cdr choices))))
                      *paths*))
                    (car choices))))))

(define fail #f)
(call/cc
 (lambda (cc)
   (set! fail
     (lambda ()
       (if (null? *paths*)
           (cc '!!failure!!)
           (let ((p1 (car *paths*)))
             (set! *paths* (cdr *paths*))
             (p1)))))))
```

Modify *choose* to support a **vector** instead of a list.

Exercise 2, Haskell

A **TextBuffer** is a simplified model of a text editor cursor. The cursor splits the text into two parts: the text **to the left** of the cursor; the text **to the right** of the cursor.

1) Define a polymorphic data type for TextBuffer.

2) Implement the following functions:

- *toList :: TextBuffer a -> [a]* Returns the complete contents of the buffer as a list.
- *fromList :: [a] -> TextBuffer a* Creates a buffer with the cursor positioned at the beginning.
- *moveLeft :: TextBuffer a -> TextBuffer a* Moves the cursor one position to the left (if possible).
- *moveRight :: TextBuffer a -> TextBuffer a* Moves the cursor one position to the right (if possible).
- *insert :: a -> TextBuffer a -> TextBuffer a* Inserts an element at the cursor position.
- *deleteLeft :: TextBuffer a -> (Maybe a, TextBuffer a)* Deletes the element immediately to the left of the cursor.
- *deleteRight :: TextBuffer a -> (Maybe a, TextBuffer a)* Deletes the element immediately to the right of the cursor.

3) Make TextBuffer an instance of **Eq**, **Functor**, and **Applicative**.

Exercise 3, Erlang

Design and implement a **task pool manager** process that distributes jobs among worker processes. The task pool should support the following operations:

- *start(N)* Spawns a pool manager process, and the manager spawns N worker processes; returns the PID of the manager.
- *submit(PoolPid, Fun)* The pool assigns the task to an available worker; the worker evaluates Fun() and sends the result back to the caller. Fun is a function of arity 0.

- *stop(PoolPid)* Terminates all workers gracefully, and terminates the manager.

Each worker must wait for a task, execute it, send the result back to the requester, and then notify the manager that it is available again.

The manager must keep track of available workers, busy workers, and pending tasks (if all workers are busy); it must also assign tasks immediately if a worker is free, queue tasks otherwise. When a worker finishes, assign the next queued task (if any).

Solutions

Ex 1

```
(define (choose choices)
  (choose-vec choices 0 (vector-length choices)))

(define (choose-vec choices i max)
  (if (= i max)
      (fail)
      (call/cc
        (lambda (cc)
          (set! *paths*
            (cons (lambda ()
                    (cc (choose-vec choices (+ 1 i) max)))
                  *paths*)))
          (vector-ref choices i))))))
```

Ex 2

```
data TextBuffer a = TB [a] [a]
-- left  = elements left of cursor (reversed)
-- right = elements right of cursor (normal order)

toList :: TextBuffer a -> [a]
toList (TB l r) = reverse l ++ r

fromList :: [a] -> TextBuffer a
fromList xs = TB [] xs

moveLeft :: TextBuffer a -> TextBuffer a
moveLeft (TB (x:xs) r) = TB xs (x:r)
moveLeft tb             = tb    -- nothing to the left

moveRight :: TextBuffer a -> TextBuffer a
moveRight (TB l (x:xs)) = TB (x:l) xs
moveRight tb            = tb    -- nothing to the right

insert :: a -> TextBuffer a -> TextBuffer a
insert x (TB l r) = TB (x:l) r

deleteLeft :: TextBuffer a -> (Maybe a, TextBuffer a)
deleteLeft (TB (x:xs) r) = (Just x, TB xs r)
deleteLeft tb            = (Nothing, tb)

deleteRight :: TextBuffer a -> (Maybe a, TextBuffer a)
deleteRight (TB l (x:xs)) = (Just x, TB l xs)
deleteRight tb            = (Nothing, tb)

instance Eq a => Eq (TextBuffer a) where
  tb1 == tb2 = toList tb1 == toList tb2

instance Functor TextBuffer where
  fmap f (TB l r) = TB (map f l) (map f r)

instance Applicative TextBuffer where
  pure x = TB [] [x]
  tf <*> tx = fromList (toList tf <*> toList tx)
```

Ex 3

```
start(N) when N > 0 ->
  spawn(fun() -> Workers = spawnw(N, []), manager_loop(Workers, #{}, []) end).

spawnw(0, Workers) -> Workers;
spawnw(N, Workers) ->
  Mgr = self(),
  spawnw(N-1, [spawn(fun() -> worker(Mgr) end) | Workers]).

submit(PoolPid, Fun) ->
  PoolPid ! {submit, self(), Fun},
  receive
    {result, Result} -> Result
  end,
  Result.

stop(PoolPid) ->
  PoolPid ! Stop, ok.
```

```

manager_loop(Available, Busy, Queue) ->
  receive
    {submit, From, Fun} ->
      case Available of
        [W | Rest] ->
          W ! {task, From, Fun},
          manager_loop(Rest, Busy#{W => true}, Queue);
        [] ->
          manager_loop(Available, Busy, Queue ++ [{From, Fun}])
      end;
    {worker_free, W} ->
      case Queue of
        [{From, Fun} | RestQueue] ->
          W ! {task, From, Fun},
          manager_loop(Available, Busy, RestQueue);
        [] ->
          manager_loop([W | Available], maps:remove(W, Busy), Queue)
      end;
    stop ->
      lists:foreach(fun(W) -> W ! stop end, Available),
      maps:foreach(fun(W,_) -> W ! stop end, Busy),
      ok
  end.

end.

worker(Manager) ->
  receive
    {task, From, Fun} ->
      Result = Fun(),
      From ! {result, Result},
      Manager ! {worker_free, self()},
      worker(Manager);
    stop -> ok
  end.

end.

```